Machine learning notes

Churui Tian

July 11, 2026

1 Machine Learning Fundamentals

1.1 What is Machine Learning

Machine learning(ML) is a subset of Artificial Intelligence(AI) where the system automatically learns pattern from data and improves their performance without being explicitly programmed.

1.2 Supervised learning vs Unsupervised learning

Supervised learning trains models using labeled data to predict outcome.
Unsupervised learning finds hidden patterns and structures in unlabeled data.

1.3 Classification and regression

Classification predicts discrete categories or labels (e.g.spam vs. not spam).
Regression predicts continuous numerical values (e.g. predicting house prices)

1.4 Train / validation / test split

training set: used to train the model.
validation set: during training to tune hyper-parameters and prevent overfitting.
testing set: used only once for evaluation of the final model.

1.5 Overfitting vs Underfitting

overfitting: The model is too complex. It learns the training data very well, but fails on the testing/evaluation data.
underfitting: The model is too simple to understand the data, so it performs poorly on both the data it was trained on and the new data it is tested on.

1.6 Basic data prepossessing

a) missing value - drop, imputation
b) categorical data - label encoding, one hot encoding

c) feature scaling - normalization, standardization
1.normalization(min-max)
2.standardization(z-score)

$$X_{\text{norm}} = \frac{X - X_{\text{min}}}{X_{\text{max}} - X_{\text{min}}} \tag{1}$$

$$X_{\text{std}} = \frac{X - \mu}{\sigma} \tag{2}$$

d) outliers - z-score, IQR

2 Model Evaluation

2.1 Classification Metrics

Table 1: confusion matrix

act pred	True	False
True	TP	FN
False	FP	TN

2.1.1 Definition

True Positive (TP): Correctly predicted positive.
True Negative (TN): Correctly predicted negative.
False Positive (FP): Incorrectly predicted positive (Type I Error).
False Negative (FN): Incorrectly predicted negative (Type II Error).

2.1.2 Accuracy

The ratio of correctly predicted observations to the total observations.

$$Accuracy = \frac{TP + TN}{TP + TN + FN + FP}$$

2.1.3 Precision

Out of all the instances the model predicted as positive, how many are actually positive?

$$Precision = \frac{TP}{TP + FP}$$

2.1.4 Recall

Out of all the actual positive instances, how many did the model correctly identify?

$$Recall = \frac{TP}{TP + FN}$$

2.1.5 F1-score

The harmonic mean of Precision and Recall. It provides a single metric that balances both.

$$F1 - score = 2 \cdot \frac{precision \cdot recall}{precision + recall}$$

2.1.6 ROC and AUC

The model first assigns a probability score to each sample. The samples are then sorted from highest to lowest score, and the algorithm iterates through them one by one. At each step, the True Positive Rate (TPR) and False Positive Rate (FPR) are calculated as coordinate points, which are connected to form a step-like ROC curve. This curve essentially serves as a "business decision menu," illustrating the model's performance across various thresholds and helping stakeholders scientifically define the final decision range based on their tolerance for false positives and false negatives. The total area under this ROC curve is the AUC (Area Under the Curve), which acts as the ultimate scorecard for the model's overall capability. Fundamentally representing the probability that the model ranks a randomly chosen positive sample higher than a randomly chosen negative one, AUC provides an objective measure of the model's discriminative power, remaining highly robust even in cases of extreme class imbalance

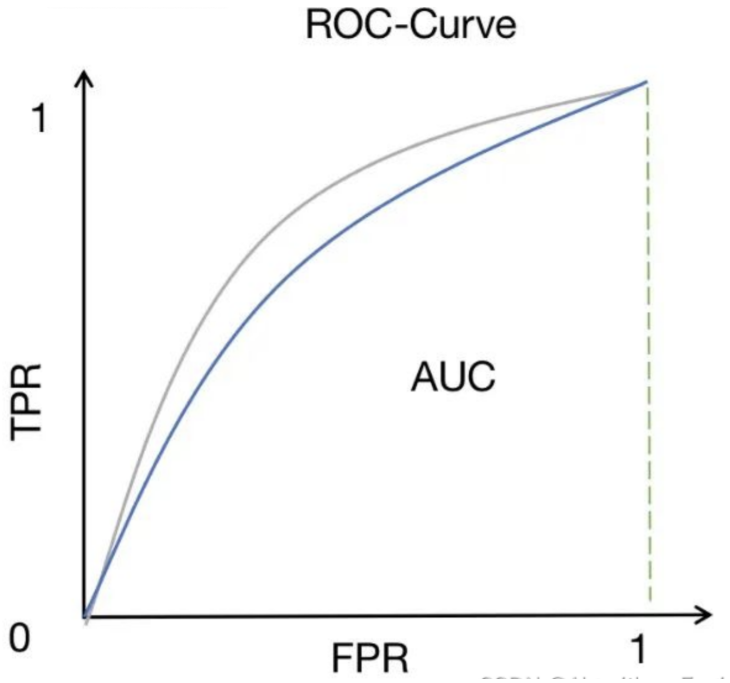

Figure 1: ROC And AUC

2.2 Regression Metrics

These are used when the model predicts continuous numerical values

2.2.1 Mean Absolute Error

On average, how far off are the predictions?

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|$$

2.2.2 Mean Squared Error

Penalizes larger errors more heavily than smaller ones (due to squaring)

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

2.2.3 Root Mean Squared Error

Brings the error metric back to the original units of the target variable, making it much easier to explain to stakeholders than MSE. Like MSE, it heavily penalizes

large errors.

$$\sqrt{MSE}$$

3 Traditional Machine Learning Models

3.1 1 R

step 1: find the category that occurs most frequently
step 2: calculate error rate
step 3: select best role: among all attributes, select the one with the lowest error rate as the final 1 R classification model.

3.1.1 Example

age	blood glucose	diabetes
≥ 60	high	yes
≥ 60	low	yes
≥ 60	low	yes
$60 \leq$	low	no

attribute: age
≥ 60: 3 yes
$60 \leq$: 1 no
error rate: $\frac{0}{3} + \frac{0}{1} = 0$
attribute: blood glucose
high: 1 yes
low: 2 yes, 1 no
error rate: $\frac{0}{1} + \frac{1}{3} = \frac{1}{3}$
we will choose age for prediction, because of lowest error rate.

3.1.2 Advantage and disadvantage

advantage: Simple and easy to understand, with excellent performance on specific datasets
disadvantage: Sensitive to the imbalance in category distribution; Limited ability to express complex tasks.

3.2 K-nearest-neighbor

3.2.1 definition

when predicting the categorical or the value of a new sample, the algorithm find the k nearest neighbors in the training dataset, and using majority vote of k neighbors or average value to obtain the result.

3.2.2 advantages and disadvantages

K-Nearest Neighbors (KNN) is a simple, intuitive, and non-parametric algorithm that requires zero training time, adapts well to non-linear data, and naturally handles multi-class problems. However, it suffers from slow prediction speed because it computes distances to every training point for each new query, struggles with high-dimensional data due to the curse of dimensionality, is highly sensitive to feature scaling and irrelevant features, requires careful tuning of the hyperparameter K to avoid overfitting or underfitting, and demands large memory storage since it keeps the entire dataset in memory. It works best on small-to-medium, clean, low-dimensional datasets where interpret-ability is more important than fast inference.

3.3 Random Forests

1. **Input:** Training set D, number of trees T, number of features to consider at each split m
2. **Output:** A Random Forest model consisting of T decision trees
3. For $t = 1$ to T:
   (a) Draw n samples with replacement (Bootstrap sampling) from D to obtain D_t
   (b) Build a decision tree using D_t:
      i. At each node, randomly select m features from all features
      ii. Choose the best split among these m features
      iii. Recursively split until stopping criteria are met (e.g., maximum depth, minimum samples per leaf, etc.)
   (c) Save the trained tree h_t
4. Return the ensemble of T trees: $\{h_1, h_2, \ldots, h_T\}$

3.4 Support vector machine

Find a hyperplane (decision boundary) that maximizes the "margin" between the two classes of data points.

1. Input training data (x_i, y_i), where $y_i \in \{-1, +1\}$, $i = 1, 2, \ldots, n$
2. Choose a kernel function $K(x_i, x_j)$ and a penalty parameter C
3. Solve the dual optimization problem to obtain α_i:

$$\max_{\alpha} \sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j y_i y_j K(x_i, x_j)$$

subject to:

$$\sum_{i=1}^{n} \alpha_i y_i = 0, \quad 0 \leq \alpha_i \leq C$$

4. Find the support vectors: points (x_i, y_i) corresponding to $\alpha_i > 0$
5. Compute the weight vector $\mathbf{w}$ and bias b:

$$\mathbf{w} = \sum_{i=1}^{n} \alpha_i y_i \phi(x_i)$$

$$b = y_k - \sum_{i=1}^{n} \alpha_i y_i K(x_i, x_k)$$

where x_k is any support vector

6. Predict the label for a new sample x:

$$f(x) = \text{sign}\left(\sum_{i=1}^{n} \alpha_i y_i K(x_i, x) + b\right)$$

where:

$$\text{sign}(z) = \begin{cases} +1, & z > 0 \\ -1, & z < 0 \end{cases}$$

3.5 Linear Regression

2-D linear regression

$$y = \beta 0 + \beta 1 \cdot x$$

3-D linear regression

$$y = \beta 0 + \beta 1 \cdot x1 + \beta 2 \cdot x2 + ... + \beta n \cdot xn$$

$$Goal : minimizelossfunction - - > Loss = sum(Ytrue - Ypred)^2$$

example

$$\begin{pmatrix} \beta 0 & x & y \\ 1 & 1 & 2 \\ 1 & 2 & 4 \end{pmatrix}$$

$findx^T$

$$\begin{pmatrix} 1 & 1 \\ 1 & 2 \end{pmatrix}$$

$find X^T X$

$$\begin{pmatrix} 2 & 3 \\ 3 & 5 \end{pmatrix}$$

$find (X^T X)^{-1}$

$$\begin{pmatrix} 5 & -3 \\ -3 & 2 \end{pmatrix}$$

$find (X^T Y)$

$$\begin{pmatrix} 6 \\ 10 \end{pmatrix}$$

$find (X^T X)^{-1} \cdot (X^T Y)$

$$\begin{pmatrix} 0 \\ 2 \end{pmatrix}$$

then, $\beta 0 = 0, \beta 1 = 2$

3.6 Logistic Regression

Logistic regression extends linear regression by mapping its output to a probability space. The computation involves three main components: a linear combination of inputs, a sigmoid transformation, and an optimization procedure.

Step 1: Linear Combination First, compute the weighted sum of the input features X:

$$z = w_1 x_1 + w_2 x_2 + \cdots + w_n x_n + b = W^T X \tag{3}$$

where $W = (w_1, w_2, \ldots, w_n)^T$ is the weight vector, and b is the bias term (intercept). The value of z ranges over $(-\infty, +\infty)$.

Step 2: Sigmoid Transformation Map z to a probability value in $(0, 1)$ using the sigmoid (logistic) function:

$$h(x) = \sigma(z) = \frac{1}{1 + e^{-z}} = \frac{1}{1 + e^{-(W^T X)}} \tag{4}$$

The output $h(x)$ represents the estimated probability that the sample belongs to the positive class (class 1). The decision rule is:

$$\hat{y} = \begin{cases} 1, & \text{if } h(x) \geq 0.5 \\ 0, & \text{if } h(x) < 0.5 \end{cases} \tag{5}$$

- As $z \to +\infty$, $h(x) \to 1$, indicating strong confidence in class 1.
- As $z \to -\infty$, $h(x) \to 0$, indicating strong confidence in class 0.
- At $z = 0$, $h(x) = 0.5$, representing maximum uncertainty.

Step 3: Loss Function Logistic regression is trained using maximum likelihood estimation. The corresponding loss function is the binary cross-entropy (log loss). For a single training sample $(x^{(i)}, y^{(i)})$, where $y^{(i)} \in \{0, 1\}$:

$$\mathcal{L}(h(x^{(i)}), y^{(i)}) = -\left[y^{(i)} \log(h(x^{(i)})) + (1 - y^{(i)}) \log(1 - h(x^{(i)}))\right] \tag{6}$$

For the entire dataset of m samples, the average cost function is:

$$J(W) = -\frac{1}{m} \sum_{i=1}^{m} \left[y^{(i)} \log(h(x^{(i)})) + (1 - y^{(i)}) \log(1 - h(x^{(i)}))\right] \tag{7}$$

This cost function is convex, guaranteeing a unique global minimum.

Step 4: Parameter Optimization via Gradient Descent To minimize $J(W)$, we compute the gradient with respect to each weight W_j. The partial derivative is:

$$\frac{\partial J}{\partial W_j} = \frac{1}{m} \sum_{i=1}^{m} \left(h(x^{(i)}) - y^{(i)}\right) x_j^{(i)} \tag{8}$$

The parameters are updated iteratively in the negative gradient direction:

$$W_j := W_j - \alpha \cdot \frac{\partial J}{\partial W_j}, \qquad \text{for } j = 1, 2, \ldots, n \tag{9}$$

where $\alpha > 0$ is the learning rate. The bias term b is updated similarly:

$$b := b - \alpha \cdot \frac{1}{m} \sum_{i=1}^{m} \left(h(x^{(i)}) - y^{(i)}\right) \tag{10}$$

The algorithm repeats these updates until convergence (i.e., when $J(W)$ stops decreasing significantly or a maximum number of iterations is reached).

Step 5: Numerical Example Consider a single training example $x = [1, 2]$ with true label $y = 1$. Initialize $W = [0, 0]$, $b = 0$, and set $\alpha = 0.1$.

Forward pass:

$$z = 0 \cdot 1 + 0 \cdot 2 = 0, \qquad h = \sigma(0) = \frac{1}{1 + e^0} = 0.5 \tag{11}$$

Loss evaluation:

$$\mathcal{L} = -[1 \cdot \log(0.5) + (1 - 1) \cdot \log(1 - 0.5)] = -\log(0.5) \approx 0.693 \tag{12}$$

Gradient computation:

$$\frac{\partial J}{\partial W_1} = (0.5 - 1) \cdot 1 = -0.5, \qquad \frac{\partial J}{\partial W_2} = (0.5 - 1) \cdot 2 = -1.0 \tag{13}$$

Parameter update:

$$W_1 := 0 - 0.1 \times (-0.5) = 0.05, \qquad W_2 := 0 - 0.1 \times (-1.0) = 0.1 \tag{14}$$

After this single update, h increases, indicating improved alignment with the true label. Repeating this process across all training samples eventually converges to the optimal parameters.

Summary Logistic regression computes a linear score $z = W^T X$, transforms it into a probability $h(x) = \sigma(z)$, and learns the parameters W and b by minimizing the cross-entropy loss using gradient descent. The decision boundary is linear, making it a simple but powerful baseline classifier.

Backpropagation (gradient computation):

$$\frac{\partial J}{\partial W_1} = (0.5 - 1) \cdot 1 = -0.5, \qquad \frac{\partial J}{\partial W_2} = (0.5 - 1) \cdot 2 = -1.0 \tag{15}$$

Weight update:

$$W_1 = 0 - 0.1 \times (-0.5) = 0.05, \qquad W_2 = 0 - 0.1 \times (-1.0) = 0.1 \tag{16}$$

3.7 K-mean

3.7.1 definition

The execution of K-mean algorithm is a continuous iterative loop. step 1: initialization
determine the number of clusters - k to be divided, and randomly select k data points as the initial clustering centers
step 2: assignment
calculate distance (usually using Euclidean distance) of each sample in the dataset to each k centroids, and assign the sample to the cluster where the centroid with the shortest distance is located.
step 3: update
recalculate the centroid of each cluster. the new centroid is the mean of all feature value of samples with that cluster.
step 4: iteration and convergence
repeatedly perform the "allocation" and "update" steps until the termination condition is met, such as maximum number of iteration is reached.

3.7.2 advantages and disadvantages

advantages: simple and efficient; highly interpretable. disadvantages: the value of k needs to be preset; sensitive to initial values as well as outliers.

Deep Learning Fundamentals

4 Deep Learning Fundamentals

4.1 Perceptron

4.1.1 Introduction

A perceptron is a supervised neural network that uses a step transfer function to produce a binary output. Its output is the weighted sum of its inputs, passed through a step function.

4.1.2 Example

Consider the following input vectors and their corresponding target outputs:

Input	Target
$p_1 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$	$t_1 = 1$
$p_2 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$	$t_2 = 0$
$p_3 = \begin{bmatrix} -1 \\ -1 \end{bmatrix}$	$t_3 = 0$

Initial conditions:

$$W = \begin{bmatrix} 0.5 & -1 \end{bmatrix}, \quad b = 1$$

Applying p_1:

$$a = \text{step}(Wp_1 + b) = \text{step}(0.5 \cdot 1 + (-1) \cdot 1 + 1) = \text{step}(0.5) = 1$$

Since $t_1 = 1$ and $a = 1$, the error is:

$$e = t_1 - a = 0$$

No update is needed.

Applying p_2:

$$a = \text{step}(Wp_2 + b) = \text{step}(0.5 \cdot 1 + (-1) \cdot 1 + 1) = \text{step}(0.5) = 1$$

Since $t_2 = 0$ and $a = 1$, the error is:

$$e = t_2 - a = -1$$

Update weights and bias:

$$W_{\text{new}} = W_{\text{old}} + e \cdot p_2^T = \begin{bmatrix} 0.5 & -1 \end{bmatrix} + (-1) \cdot \begin{bmatrix} 1 & 1 \end{bmatrix} = \begin{bmatrix} -0.5 & -2 \end{bmatrix}$$

$$b_{\text{new}} = b_{\text{old}} + e = 1 + (-1) = 0$$

Applying p_3:

$$a = \text{step}(Wp_3 + b) = \text{step}((-0.5)(-1) + (-2)(-1) + 0) = \text{step}(2.5) = 1$$

Since $t_3 = 0$ and $a = 1$, the error is:

$$e = t_3 - a = -1$$

Update weights and bias:

$$W_{\text{new}} = W_{\text{old}} + e \cdot p_3^T = \begin{bmatrix} -0.5 & -2 \end{bmatrix} + (-1) \cdot \begin{bmatrix} -1 & -1 \end{bmatrix} = \begin{bmatrix} 0.5 & -1 \end{bmatrix}$$

$$b_{\text{new}} = b_{\text{old}} + e = 0 + (-1) = -1$$

4.1.3 Perceptron Learning Rule Summary

The perceptron weight update rule is:

$$\boxed{W_{\text{new}} = W_{\text{old}} + e \cdot p^T, \quad b_{\text{new}} = b_{\text{old}} + e}$$

where $e = t - a$ is the error, p is the input vector, and a is the actual output.

4.2 Limitation of Single-Layer Perceptron

A single-layer perceptron can only classify **linearly separable** data. It fails on problems like XOR, where the data points are not linearly separable.

5 Backpropagation Algorithm

5.1 From Single-Layer to Multi-Layer Perceptrons

5.1.1 Parameter Updates in Single-Layer Perceptrons

A single-layer perceptron can be expressed as:

$$\hat{y} = \sigma(z), \quad z = \sum_{i=1}^{n} w_i x_i + b = W^T x + b$$

where $\sigma(\cdot)$ is the Sigmoid activation function:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

We adopt the Mean Squared Error (MSE) as the loss function:

$$L = \frac{1}{2}(y - \hat{y})^2$$

The goal is to update each weight w_i to minimize the loss L. Using gradient descent:

$$w_i \leftarrow w_i - \eta \frac{\partial L}{\partial w_i}$$

where η is the learning rate.

Since L depends on w_i indirectly through $\hat{y}$ and z, we apply the chain rule:

$$\frac{\partial L}{\partial w_i} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z} \cdot \frac{\partial z}{\partial w_i}$$

We compute each term explicitly:

$$\frac{\partial L}{\partial \hat{y}} = \hat{y} - y, \qquad \frac{\partial \hat{y}}{\partial z} = \hat{y}(1 - \hat{y}), \qquad \frac{\partial z}{\partial w_i} = x_i$$

Substituting back, we obtain the gradient:

$$\frac{\partial L}{\partial w_i} = (\hat{y} - y) \cdot \hat{y}(1 - \hat{y}) \cdot x_i$$

Thus, the update rule for a single-layer perceptron is:

$$\boxed{w_i \leftarrow w_i - \eta \cdot (\hat{y} - y) \cdot \hat{y}(1 - \hat{y}) \cdot x_i}$$

5.1.2 Extension to Multi-Layer Perceptrons

In a multi-layer perceptron (MLP) with one hidden layer, the forward propagation consists of:

$$z_1 = W_1 x + b_1 \quad \text{(hidden layer linear transform)} \tag{17}$$
$$h = \text{ReLU}(z_1) \quad \text{(hidden layer activation)} \tag{18}$$
$$z_2 = W_2 h + b_2 \quad \text{(output layer linear transform)} \tag{19}$$
$$\hat{y} = \sigma(z_2) \quad \text{(output layer activation)} \tag{20}$$

where $\text{ReLU}(z) = \max(0, z)$.
The loss remains:

$$L = \frac{1}{2}(y - \hat{y})^2$$

5.2 Backpropagation for Multi-Layer Perceptrons

The key idea of backpropagation is to propagate the error gradient backward from the output layer to the hidden layer, applying the chain rule repeatedly.

5.2.1 Output Layer Gradients

Define the output layer error term δ_2 as:

$$\delta_2 = \frac{\partial L}{\partial z_2} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z_2} = (\hat{y} - y) \cdot \hat{y}(1 - \hat{y})$$

The gradients for the output layer weights and bias are:

$$\frac{\partial L}{\partial w_j^{(2)}} = \delta_2 \cdot h_j, \qquad \frac{\partial L}{\partial b_2} = \delta_2$$

Therefore, the update rules for the output layer are:

$$w_j^{(2)} \leftarrow w_j^{(2)} - \eta \cdot \delta_2 \cdot h_j \tag{21}$$
$$b_2 \leftarrow b_2 - \eta \cdot \delta_2 \tag{22}$$

5.2.2 Hidden Layer Gradients (Error Backpropagation)

To update the hidden layer weights, we must compute how the loss L depends on the hidden layer output h_j.

First, compute the derivative of L with respect to h_j:

$$\frac{\partial L}{\partial h_j} = \frac{\partial L}{\partial z_2} \cdot \frac{\partial z_2}{\partial h_j} = \delta_2 \cdot w_j^{(2)}$$

Next, compute the derivative of h_j with respect to z_{1j} (the ReLU derivative):

$$\frac{\partial h_j}{\partial z_{1j}} = \begin{cases} 1, & z_{1j} > 0 \\ 0, & z_{1j} \leq 0 \end{cases} = \mathbb{1}_{z_{1j}>0}$$

Define the hidden layer error term δ_{1j}:

$$\delta_{1j} = \frac{\partial L}{\partial z_{1j}} = \frac{\partial L}{\partial h_j} \cdot \frac{\partial h_j}{\partial z_{1j}} = \delta_2 \cdot w_j^{(2)} \cdot \mathbb{1}_{z_{1j}>0}$$

In vector form:

$$\boxed{\delta_1 = (\delta_2 \cdot W_2^T) \odot \mathrm{ReLU}'(z_1)}$$

where $\odot$ denotes element-wise multiplication (Hadamard product), and $\mathrm{ReLU}'(z_1)$ is the derivative of ReLU applied element-wise.

The gradients for the hidden layer weights and biases are:

$$\frac{\partial L}{\partial w_{ji}^{(1)}} = \delta_{1j} \cdot x_i, \qquad \frac{\partial L}{\partial b_{1j}} = \delta_{1j}$$

Thus, the update rules for the hidden layer are:

$$w_{ji}^{(1)} \leftarrow w_{ji}^{(1)} - \eta \cdot \delta_{1j} \cdot x_i \tag{23}$$

$$b_{1j} \leftarrow b_{1j} - \eta \cdot \delta_{1j} \tag{24}$$

5.3 Summary of the Backpropagation Algorithm

The complete backpropagation algorithm for an MLP with one hidden layer can be summarized as follows:

1. **Forward pass**: Compute z_1, h, z_2, and $\hat{y}$.
2. **Compute output error**: $\delta_2 = (\hat{y} - y) \cdot \hat{y}(1 - \hat{y})$.
3. **Update output layer**:
$$W_2 \leftarrow W_2 - \eta \cdot \delta_2 \cdot h^T, \quad b_2 \leftarrow b_2 - \eta \cdot \delta_2$$
4. **Backpropagate error**:
$$\delta_1 = (\delta_2 \cdot W_2^T) \odot \mathrm{ReLU}'(z_1)$$
5. **Update hidden layer**:
$$W_1 \leftarrow W_1 - \eta \cdot \delta_1 \cdot x^T, \quad b_1 \leftarrow b_1 - \eta \cdot \delta_1$$

5.4 Comparison: Single-Layer vs. Multi-Layer Updates

Single-Layer Perceptron	**Multi-Layer Perceptron**
Compute $\delta = (\hat{y} - y) \cdot \hat{y}(1 - \hat{y})$	Compute $\delta_2 = (\hat{y} - y) \cdot \hat{y}(1 - \hat{y})$
Update $w_i \leftarrow w_i - \eta \cdot \delta \cdot x_i$	Update $W_2 \leftarrow W_2 - \eta \cdot \delta_2 \cdot h^T$
	Backpropagate: $\delta_1 = (\delta_2 \cdot W_2^T) \odot \text{ReLU}'(z_1)$
	Update $W_1 \leftarrow W_1 - \eta \cdot \delta_1 \cdot x^T$

5.5 Key Insight

The backpropagation algorithm is a direct extension of the single-layer perceptron update rule. In both cases, we apply the chain rule to compute gradients. The difference is that in MLPs, the error must be **propagated backward** through each layer, allowing every weight in the network to be updated based on its contribution to the final prediction error.

$$\boxed{\text{Backpropagation} = \text{Chain Rule} + \text{Gradient Descent}}$$